

REALTY360 AUSTIN

Application Architecture Report

System design, data flows, and engineering decisions
for Travis County property analysis

Version 1.3

July 7, 2026 | Austin / Travis County, TX

MLS SQLite

TCAD Neon Cache

Unified Comps

Flip Engine

Policy B Tax

Technology stack

- Client: Vite + React + shadcn/ui
- API: Express (TypeScript)
- Domain: shared/ - comps, flip, TCAD, property-profile
- Data: data/mls.sqlite + Neon tcad_parcel

Contents

1	Executive Summary
2	System Architecture
3	Repository Layout
4	Data Sources & Storage
5	Core User Flows
6	API Surface
7	Domain Modules
8	Flip Prediction Engine
9	Key Design Decisions
10	Operational Commands
11	Roadmap & Gaps

Diagrams are embedded inline throughout the report. The markdown appendix uses ASCII placeholders; this PDF renders vector architecture diagrams instead.

- 1 Executive Summary
- 2 System Architecture
- 3 Repository Layout
- 4 Data Sources & Storage
- 5 Core User Flows
- 6 API Surface
- 7 Domain Modules
- 8 Flip Prediction Engine
- 9 Key Design Decisions
- 10 Operational Commands
- 11 Roadmap & Gaps
- 12 Appendix: Diagrams

1. Executive Summary

Realty360 Austin is a full-stack property analysis tool for Travis County real estate investors and analysts. It combines:

- MLS closed and open listings (sale comparables, subject lookup)
- TCAD tax parcel data (assessed/appraised values, parcel centroids)
- Unified comparables search (closed sales, active listings, optional tax-reference parcels; MLS comps enriched with TCAD tax fields)
- Property profile composition with in-process cache and multi-parcel TCAD (taxCandidates)
- Post-flip deal analysis (ARV from comps, rehab tiers, hold/financing costs with Policy B multi-parcel tax, margin viability)

The app is built as Vite + React (client) and Express (API), with shared TypeScript logic in shared/. Production serves a single Node process; development runs Vite on port 3000 proxying /api to Express on 3001.

What it is not (yet): a production MLS Grid proxy for the browser, a machine-learning ARV model, or a forced MLS<->TCAD crosswalk. Those are documented future phases.

2. System Architecture

2.1 High-Level Topology

SYSTEM TOPOLOGY

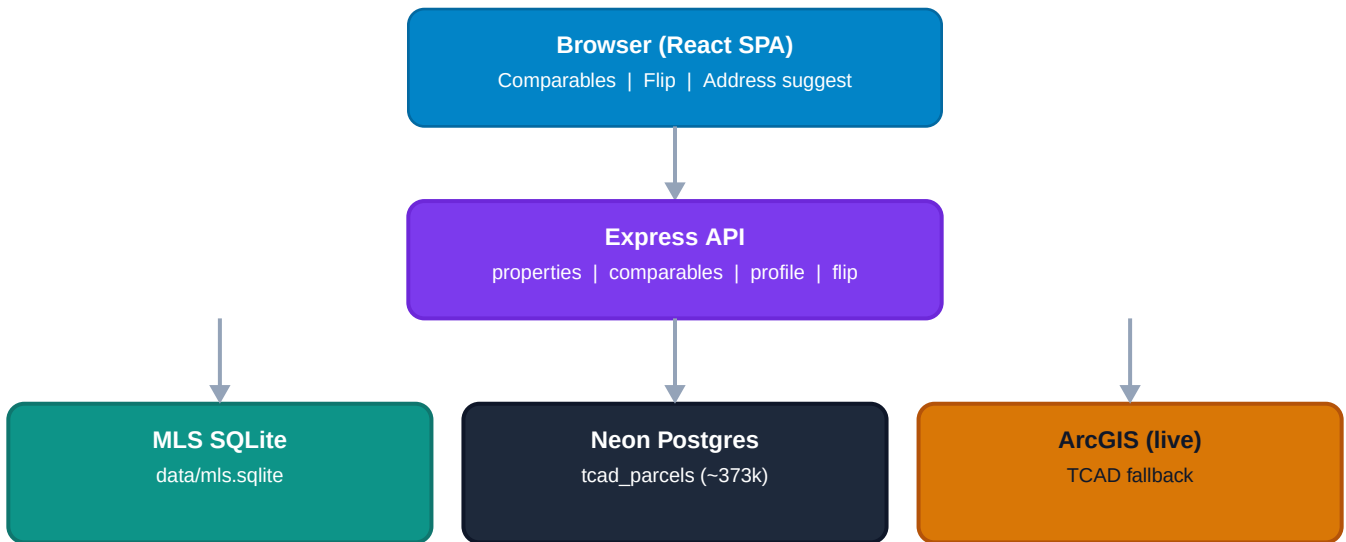


Figure 1 - Browser, API, and data store layers

2.2 Request Path (Development)

DEVELOPMENT REQUEST PATH

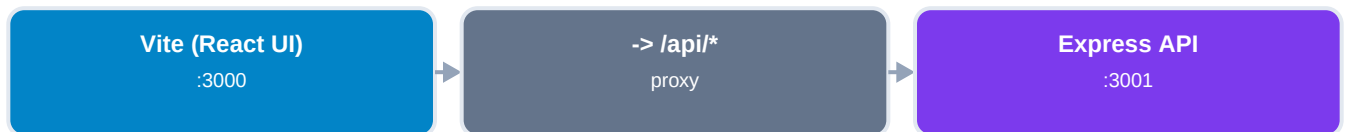


Figure 2 - Development ports and proxy path

Component	Port	Role
Vite dev server	3000	UI, proxies /api -> 3001
Express dev:api	3001	JSON API, reads SQLite + Postgres
pnpm start (prod)	3000	Bundled static + API (or API-only if no build)

2.3 Shared Logic

Business rules live in shared/ so the same comp-matching, flip math, TCAD address scoring, and DTO composition run in scripts, API routes, and (where applicable) the client via @shared alias.

3. Repository Layout

Path	Purpose
client/	React UI (Comparables landing, flip results, shadcn components)
server/	Express app, route handlers
shared/	Domain logic: MLS, TCAD, comparables, property-profile, flip
shared/env/load-env-local.ts	Loads .env.local for Node scripts and Express
shared/mls/sqlite.ts	MLS SQLite schema and query helpers (runtime)
scripts/	Seed jobs (seed-mls, seed-tcad), smoke-flip, architecture PDF
data/	Gitignored runtime data (mls.sqlite, raw JSON dumps)
docs/	Roadmap and architecture documentation

4. Data Sources & Storage

4.1 MLS (Multiple Listing Service)

Aspect	Detail
Upstream	MLS Grid API (https://api-demo.mlsgrid.com/v2 or production URL)
Originating system	actris (Austin Board of Realtors) - <code>shared/mls/constants.ts</code>
Local store	SQLite <code>data/mls.sqlite</code> , table <code>properties</code>
Seed command	<code>pnpm seed:mls</code> (closed + active/pending residential)
Runtime	API reads SQLite only; no live MLS Grid on user requests
Token	<code>MLS_GRID_TOKEN</code> or <code>VITE_MLS_GRID_TOKEN</code> in <code>.env.local</code> (seed only)

Row shape: normalized address, lat/lon, beds/baths/sqft, list/close price, close date, pool, garage, lot acres, property condition, status.

4.2 TCAD (Travis Central Appraisal District)

TCAD READ PATH

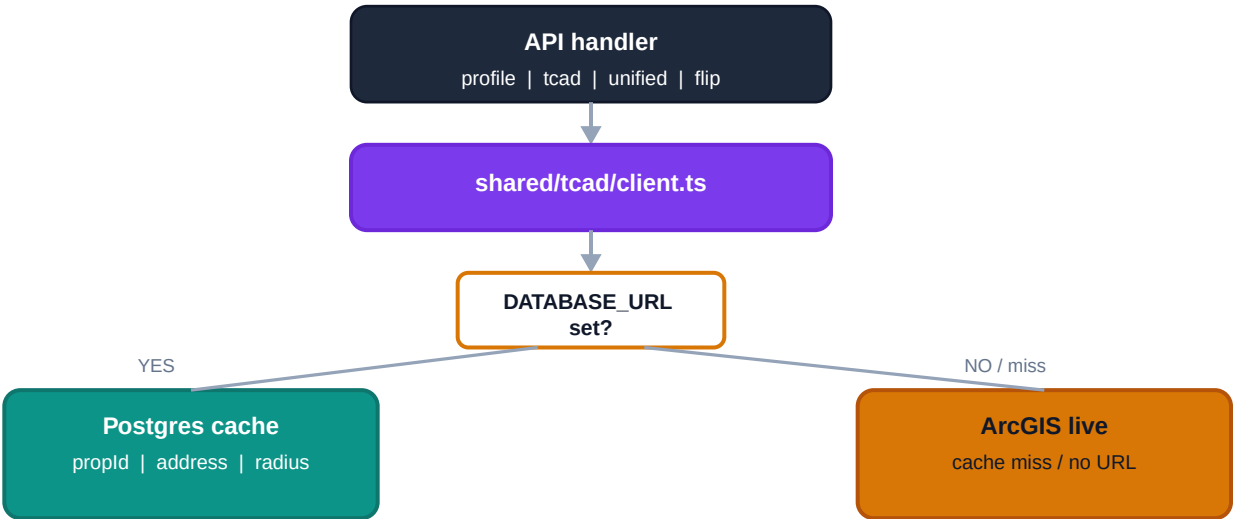


Figure 7 - Cache-first TCAD read path

Aspect	Detail
Upstream	ArcGIS MapServer layer TCAD/MapServer/0 (full county parcels)
Local store	Postgres tcad_parcel (Neon) via DATABASE_URL
Seed command	pnpm seed:tcad (~387k parcels, paginated ETL)
Runtime	Cache-first in shared/tcad/client.ts; live ArcGIS fallback
Coverage	Travis County only - Williamson/Georgetown addresses return not_found

Fields cached: PROP_ID, geo_id, situs address, tax values (appraised, market, assessed), acres, WGS-84 centroid.

4.3 Data Independence Principle

MLS and TCAD are intentionally separate. There is no required listing_key <-> prop_id crosswalk in v1. Profiles compose both sources at request time when possible; partial profiles are valid.

5. Core User Flows

5.1 Comparables Search

MLS COMP TCAD ENRICHMENT

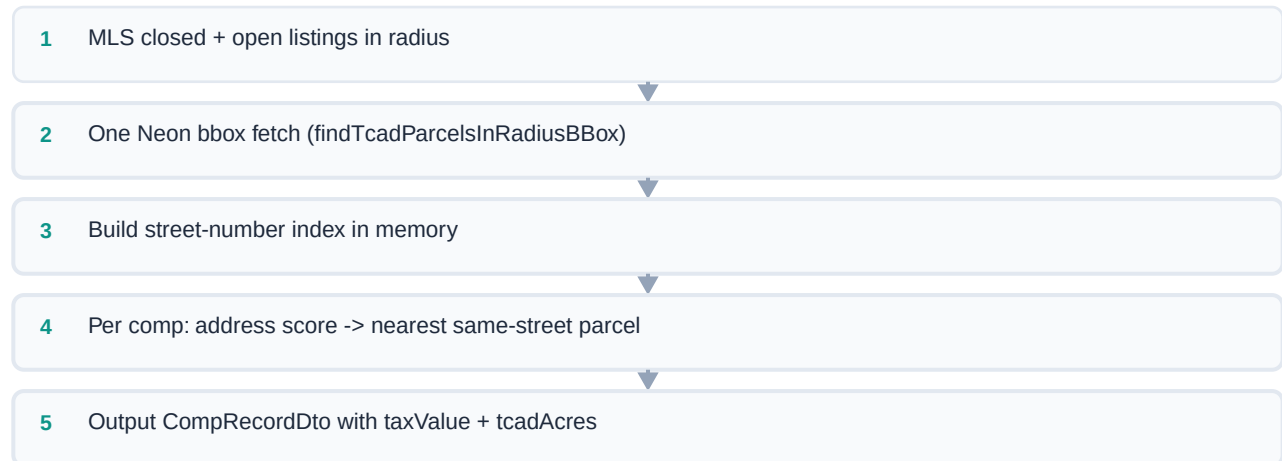


Figure 4 - Batch TCAD enrichment for MLS comps

- 1 User enters address (with typeahead from MLS SQLite).
- 2 UI calls GET /api/comparables/unified with radius, recency, optional includeTcad, optional propId.
- 3 Server loads subject via fetchPropertyProfileCached (MLS + TCAD profile-first); resolves MLS row for comp filters.
- 4 Returns sections:
 - Closed sales (MLS) - sale comps, each enriched with TCAD taxValue and tcadAcres when a parcel matches
 - Active listings (MLS open status) - same TCAD enrichment on listing comps
 - Tax references (TCAD radius, includeTcad=true) - reference only, not sale comps

1 Response includes subjectProfile for subject card tax display and flip cache reuse.

Subject card: shows TCAD tax value, acres, deed date when single match; lists all taxCandidates with combined tax when ambiguous; optional parcel picker re-runs search with propId.

TCAD enrichment (closed + open comps): one Postgres radius prefetch per search

(findTcadParcelsInRadiusBBox), then in-memory address scoring and street-aware nearest-parcel fallback.

Coords for matching prefer MLS lat/lon; TCAD centroid fills gaps. UI citation: MLS + TCAD when enriched.

Comps are bucketed into exclusive mile rings (0-0.5, 0.5-1, 1-2 mi, ...) with match % from beds/baths/distance.

5.2 Property Profile

PROPERTY PROFILE COMPOSITION

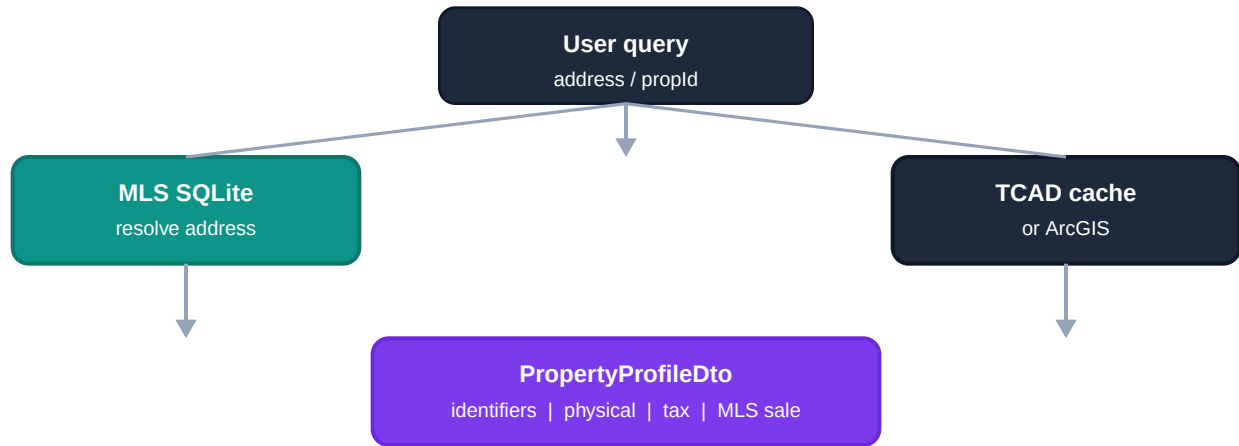


Figure 5 - MLS + TCAD profile composition

MULTI-PARCEL TCAD HANDLING

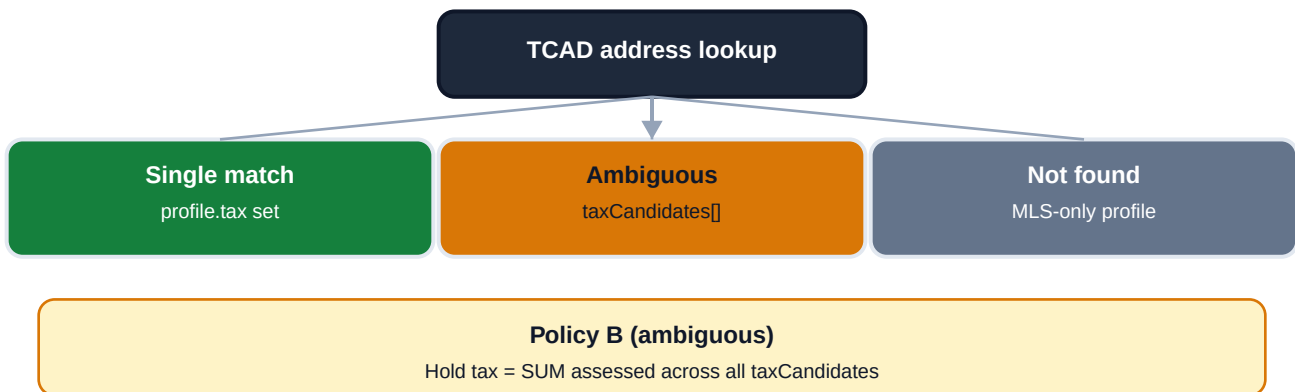


Figure 6 - Multi-parcel TCAD outcomes and Policy B

- 1 GET /api/property/profile?address= or ?propId= (or both)
- 2 fetchPropertyProfile() / fetchPropertyProfileCached() loads MLS by address and TCAD by address or propId.
- 3 composePropertyProfile() merges identifiers, physical, MLS sale, tax, location.
- 4 TCAD is always attempted for tax data even when MLS supplies coordinates.
- 5 Single TCAD match -> tax set, tcadMatch.status: "single".
- 6 Ambiguous TCAD match -> tax: null, taxCandidates[], tcadMatch.status: "ambiguous" (e.g. duplex - multiple prop_id at same situs).
- 7 No TCAD match -> MLS-only profile, tcadMatch.status: "not_found"; comparables and flip continue without tax fields.
- 8 Address matching tolerates MLS-style queries (504 Bramble Dr, Austin, TX 78745) by excluding city/ state tokens from situs scoring (shared/tcad/address-query.ts).

Profile cache: in-process Map, 30 min TTL, keys address:{norm} and/or propId:{id}address:{norm}.

Comparables search warms cache; flip reuses same profile without re-fetching TCAD.

5.3 Flip Analysis

- 1 User runs comparables search, switches to Flip analysis tab.
 - 2 Enters purchase price and rehab scope tier (cosmetic / moderate / full).
 - 3 POST /api/predict/flip in enriched mode:
 - Loads property profile via fetchPropertyProfileCached (address only - preserves taxCandidates for Policy B even if UI selected a propId for display) - Derives ARV from similar closed MLS comps (median \$/sqft x subject sqft) - Computes rehab, closing, hold, financing - Hold tax basis (Policy B): single parcel -> assessed -> appraised; ambiguous -> sum assessed across all taxCandidates; no TCAD -> purchase price - Returns viability band: strong / marginal / weak / negative
- Manual mode: user supplies explicit arv + livingAreaSqft without subject lookup.
-

6. API Surface

Method	Endpoint	Purpose
GET	/api/health	Liveness check
GET	/api/properties/suggest	Address typeahead (MLS)
GET	/api/properties/by-address	Single MLS property
GET	/api/properties/by-radius	Deprecated - closed comps only; use /api/comparables/unified
GET	/api/comparables/unified	Closed + open + optional TCAD; optional propld; returns subjectProfile
GET	/api/property/profile	Composed MLS + TCAD profile
GET	/api/tcad/property	TCAD parcel by propld or address
POST	/api/predict/flip	Rules-based flip prediction

All routes are mounted in server/app.ts.

Deprecation: GET /api/properties/by-radius responds with Deprecation: true, a Warning header, and a deprecation object in the JSON body. Prefer GET /api/comparables/unified for all new integrations.

7. Domain Modules

7.1 shared/mls/

- RESO property types, MLS Grid client, address normalization
- shared/mls/sqlite.ts: SQLite schema, radius queries, address resolution

7.2 shared/tcad/

- ArcGIS client with retry/timeout
- Address fuzzy matching (address-query.ts)
- Postgres cache (db.ts)
- Parcel centroids from polygon geometry

7.3 shared/comparables/

- match-score.ts - mile rings, match %, distance score
- comp-record.ts - unified CompRecordDto with source, compRole, taxValue, tcadAcres
- tcad-enrichment.ts - batch TCAD prefetch + in-memory match for MLS sale/listing comps
- unified-search.ts - orchestrates MLS + TCAD comp sections and enrichment

7.4 shared/property-profile/

- fetch-profile.ts - enrichment orchestration (MLS + TCAD, ambiguous handling)
- compose.ts - merges partial MLS/TCAD into PropertyProfileDto
- profile-cache.ts - in-process TTL cache keyed by address and/or propId
- tcad-tax.ts - pickProfileTaxValue, taxHoldBasisFromProfile (Policy B sum for multi-parcel)
- Provenance and completeness scoring

7.5 shared/flip/

- config.ts - Travis County rehab tiers, closing %, hold/financing defaults
 - arv-comps.ts - similarity filter, tier-based \$/sqft slice (top half / quartile)
 - deal-costs.ts - deterministic cost stack
 - predict-flip.ts - end-to-end prediction pipeline
-

8. Flip Prediction Engine

8.1 ARV (After Repair Value)

Source: MLS closed sales within radius, filtered by:

- Recency (maxAgeMonths on close_date)
- Similarity: +/-25% sqft, beds/baths within delta rules
- Rehab tier slice: cosmetic uses all similar; moderate uses top 50% \$/sqft; full uses top 25%

```
ARV = median(pricePerSqft of slice) x subjectLivingAreaSqft
```

Override: request body arv -> manual mode.

8.2 Cost Stack

FLIP PREDICTION PIPELINE

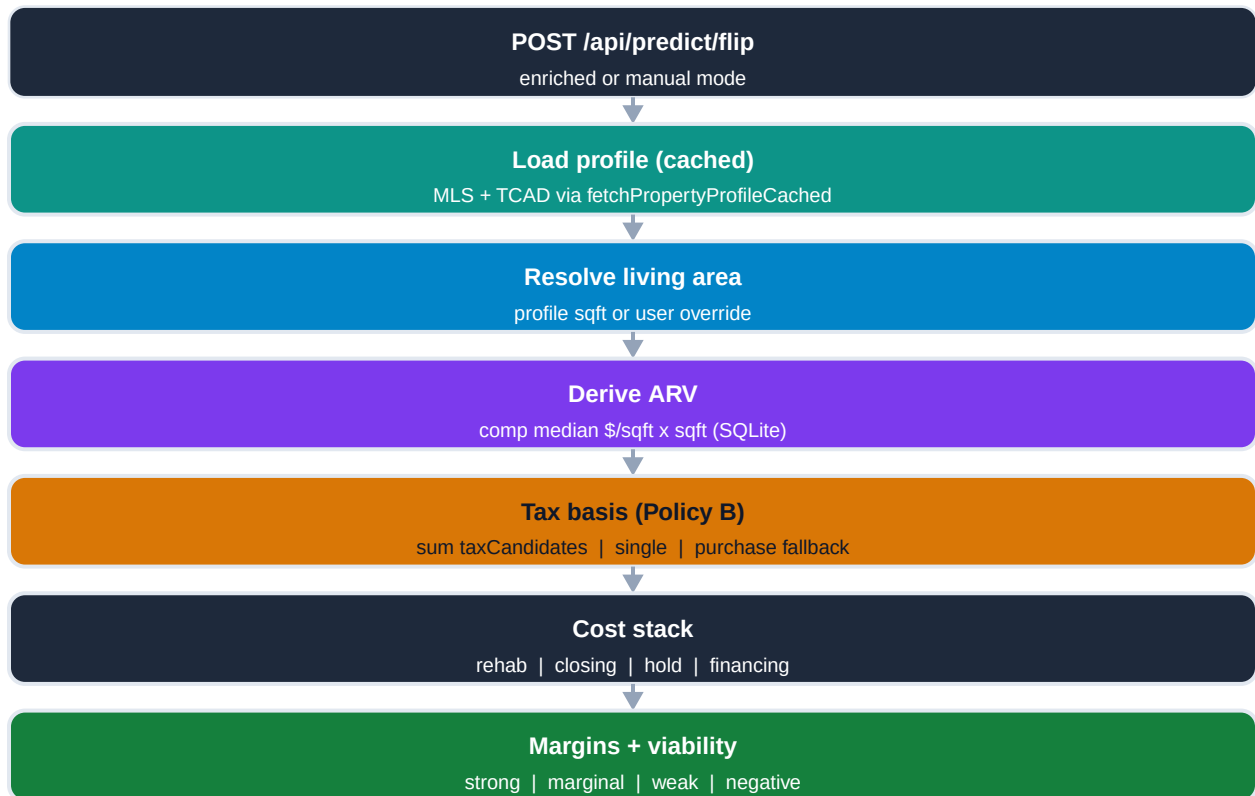


Figure 8 - Flip prediction pipeline

Component	Basis
Rehab	tier \$/sqft x living area (sqft from MLS profile)
Buy closing	2.5% of purchase price (user input)
Hold - property tax	Policy B: sum assessed across taxCandidates when ambiguous; else single assessed -> appraised -> purchase price; x 2.2% annual x hold months
Hold - insurance / utilities / HOA	fixed monthly amounts from deal config
Financing	hard-money style: LTC ratio, points, interest during hold
Sell closing	7% of ARV (ARV from MLS comps)

8.3 Viability Bands

FLIP VIABILITY BANDS

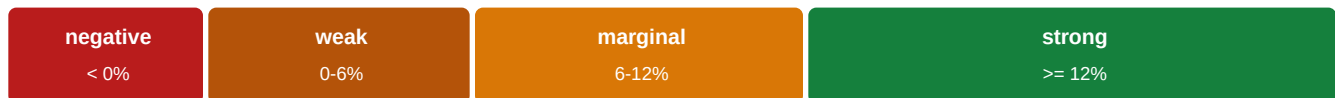


Figure 9 - Net margin viability bands

Net margin % (of ARV)	Label
>= 12%	strong
6-12%	marginal
0-6%	weak
< 0%	negative

8.4 Explicit Non-Goals

- TCAD appraised value is not used as ARV or sale comp
- Tax values affect hold cost only (when TCAD resolves)

9. Key Design Decisions

Decision	Rationale
MLS in SQLite, not live API	Fast radius queries; avoids MLS Grid rate limits on every request
TCAD in Postgres (Neon)	~373k parcels; eliminates per-request ArcGIS latency; one bbox query per comp search
Cache-first TCAD with ArcGIS fallback	Works before seed completes; resilient to cache gaps; `source: tcad-cache \tcad-arcgis`
MLS comp TCAD enrichment	Sale/listing comps show unified tax fields without merging TCAD into ARV math
Profile cache (30 min TTL)	Comparables warms profile; flip reuses without duplicate TCAD/MLS fetches
Multi-parcel Policy B	Duplex/multi-prop_id situs: hold tax sums all candidate assessed values; UI lists each parcel
MLS-only fallback	TCAD not_found -> valid MLS profile; hold tax uses purchase price
MLS-style address scoring	City/state stripped from TCAD situs match so profile + flip resolve full formatted addresses
Batch comp enrichment	Single radius prefetch + in-memory index (not N DB round-trips per comp)
No forced MLS<->TCAD link in v1	Address matching is messy; partial profiles OK; crosswalk is Phase 5 optional
TCAD tax comps are reference only	Appraised `` market sale price; prevents misleading ARV
Rules engine before ML	Auditable math; same API contract for future Python model
Shared TypeScript domain layer	One source of truth for comps, flip, and profile logic
Enriched vs manual flip modes	Supports full subject lookup or spreadsheet-style what-if
Dedupe PROP_ID on TCAD seed	ArcGIS pages can repeat prop_ids (multipart parcels)

10. Operational Commands

```
pnpm install
pnpm seed:mls          # MLS closed + active/pending -> data/mls.sqlite
pnpm seed:tcad         # Full county -> Neon tcad_parcel
pnpm dev:api           # API :3001
pnpm dev               # UI :3000
pnpm smoke:flip        # API/engine regression checks
pnpm docs:architecture-pdf # Regenerate docs/ARCHITECTURE_REPORT.pdf
pnpm build && pnpm start # Production-style single server
```

Environment (.env.local):

- VITE_MLS_GRID_TOKEN / MLS_GRID_TOKEN - MLS Grid bearer token
- DATABASE_URL - Neon Postgres for TCAD cache (tcad_parcel). Must match the Neon project/branch you intend (verify ep-... hostname in connection string vs dashboard).

11. Roadmap & Gaps

Completed (Phases 1-4 + TCAD platform)

- MLS schema extensions, mile-bucket comps, unified comparables API
- Property profile composition, TCAD propId/address APIs, profile cache, multi-parcel taxCandidates
- Subject card TCAD tax display; optional propId parcel picker
- Flip rules engine v0, flip UI on comparables landing (FlipPredictionResults)
- Policy B hold tax (taxHoldBasisFromProfile - sum assessed for ambiguous parcels)
- TCAD Neon ETL (pnpm seed:tcad, ~373k parcels), cache-first reads (shared/tcad/db.ts)
- MLS sale/listing comp enrichment with TCAD tax value + acres (tcad-enrichment.ts)
- MLS-style TCAD address matching (city/state excluded from scoring)
- Flip smoke tests (pnpm smoke:flip)

MLS <-> TCAD coverage (July 2026 audit)

MLS <-> TCAD ADDRESS COVERAGE (6,139 MLS ADDRESSES)



Figure 10 - MLS address crosswalk outcomes

Of 6,139 distinct MLS addresses crosswalked against tcad_parcels:

Outcome	Count
Single TCAD parcel	1,573
No TCAD match (MLS-only)	4,539
Ambiguous (multi-parcel)	27

Example multi-parcel test: 1613 W Braker Ln #B, Austin, TX 78758 (2 TCAD parcels, same situs). Example MLS-only: 507 Hammack Dr (MLS present; TCAD situs numbering gap 505->600 on Hammack).

Planned

Item	Description
MLS -> TCAD crosswalk	Precomputed listing_key <-> prop_id links for faster comp joins
Deal ledger	Labeled flips for ML training
Python ML ARV model	Same POST /api/predict/flip contract
geold lookup	Alternate TCAD key
Coordinate-based TCAD fallback	When address not_found but MLS lat/lon exists, match nearest parcel(s) in small radius

Monthly TCAD refresh	Cron re-run seed:tcad
----------------------	-----------------------

Known Limitations

- MLS seed cap (~2k-5k rows configurable) - not full ACTRIS history
 - Non-Travis addresses have MLS data but no TCAD tax
 - ~74% of MLS addresses have no TCAD situs match in current cache (address gaps, situs numbering mismatches)
 - Multi-parcel ambiguous is rare (~27 MLS addresses); most duplexes either score to one winner or have no TCAD situs
 - Condos/townhomes may lack unit-level TCAD situs - nearest same-street parcel used for comp enrichment
 - Flip ARV requires sufficient similar closed comps in radius
 - Home page calculator may still use embedded constants if MLS token unset
-

